

RAG Architecture and Comparative Analysis

Understanding the RAG Pipeline and Its Differentiation from Other Techniques

1. Understanding the RAG Architecture

The RAG architecture is structured around three core phases: Ingestion, Retrieval, and Generation. Each phase plays a distinct and essential role in enabling the system to produce accurate, context-aware responses based on external knowledge. The diagram below illustrates how these phases connect and interact in a complete RAG pipeline.

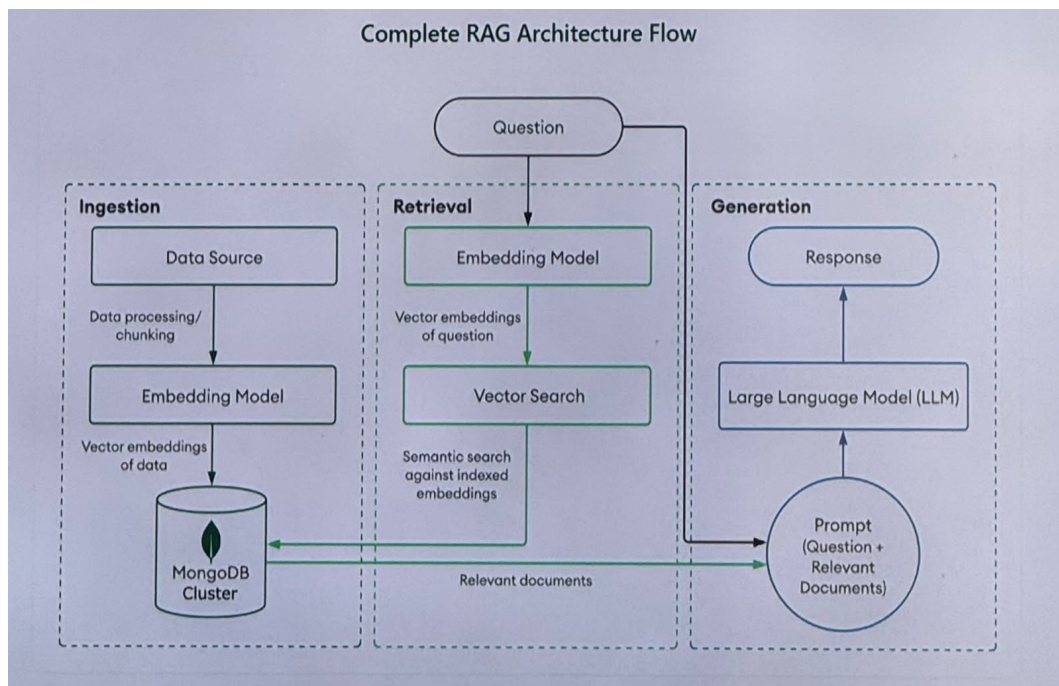


Figure 1: Complete RAG Architecture Flow

2. The Three Phases of RAG Architecture

2.1 Phase 1 — Ingestion

The Ingestion phase is a one-time setup process that runs before any queries are handled. It begins with raw data from various sources such as PDF documents, web pages, databases, or internal company files. This data is first processed and broken down into smaller, manageable pieces called chunks through a process known as data processing and chunking.

Each chunk is then passed through an Embedding Model, which converts the text into a high-dimensional numerical representation called a vector embedding. These vector embeddings capture the semantic meaning of the text, meaning chunks with similar meanings will have embeddings that are close to each other in vector space. These embeddings are stored in a MongoDB Cluster (or any vector database), which serves as the external knowledge

base for the RAG system.

2.2 Phase 2 — Retrieval

The Retrieval phase is activated every time a user submits a question. The incoming question is first passed through the same Embedding Model used during ingestion. This converts the question into a vector embedding that captures its semantic intent.

This question embedding is then used to perform a Vector Search against the indexed embeddings stored in the MongoDB Cluster. The vector search performs a semantic similarity search, meaning it finds the stored chunks whose embeddings are most similar to the question embedding, not just those that contain the same keywords. The top matching chunks, referred to as relevant documents, are retrieved and passed forward.

2.3 Phase 3 — Generation

The Generation phase takes the retrieved relevant documents and combines them with the original question to form a structured Prompt. This prompt contains both the user's question and the factual context retrieved from the knowledge base.

This enriched prompt is then fed into the Large Language Model (LLM). The LLM uses the provided context to generate a grounded, accurate response. The final output is the Response delivered back to the user. Because the LLM is given relevant source material to work with, it is far less likely to produce incorrect or hallucinated information.

3. Comparison: Prompt Engineering vs Fine-Tuning vs RAG

These three techniques represent the primary methods of adapting or directing LLM behavior. Understanding their differences helps in choosing the right approach for a given application.

Aspect	Prompt Engineering	Fine-Tuning	RAG
Definition	Crafting specific input prompts to guide the LLM towards desired outputs	Retraining a pre-trained LLM on a domain-specific dataset to update model weights	Augmenting LLM with external knowledge retrieval at query time
Knowledge Type	Uses only the model's existing training knowledge	Embeds new knowledge into model parameters during training	Dynamically retrieves fresh knowledge from external sources
Data Freshness	Static — limited to training cutoff date	Static — knowledge fixed after fine-tuning is complete	Dynamic — knowledge base can be updated at any time
Cost	Very low — no training required, just prompt design	Very high — requires GPU compute, curated datasets, and time	Moderate — needs vector DB setup but no model retraining

Aspect	Prompt Engineering	Fine-Tuning	RAG
Transparency	Low — cannot cite sources for claims	Low — knowledge is absorbed into weights, not traceable	High — can cite exact source documents used for each response
Best Use Case	Simple task direction, format control, zero- shot reasoning	Domain specialization with stable, well-defined knowledge	Dynamic knowledge bases, real-time data, custom documents
Hallucination Risk	High — model relies on potentially outdated training data	Moderate — reduces hallucination in trained domain	Low — grounded in retrieved factual content
Scalability	High — works on any task with right prompt design	Low — each domain needs separate fine-tuning runs	High — update knowledge base without modifying the model
Latency	Lowest — direct model call with no retrieval step	Low — model call only, no retrieval step	Slightly higher — includes retrieval step before generation

The RAG architecture offers the most practical balance between cost, accuracy, and flexibility for production AI systems where knowledge must remain current. While prompt engineering is sufficient for simple tasks and fine-tuning excels in narrow, static domains, RAG stands out as the preferred choice for dynamic, knowledge-intensive applications.